ТЕХНИЧЕСКИЙ РАЗБОР · СЕНТЯБРЬ 2026

Автономная ферма для решения task-based CTF

Инфраструктура, роевая координация и оптимизация потоков агентов на базе роя моделей **DeepSeek-V4**. Стратегия, экономика и инженерный блюпринт под режим живых jeopardy-соревнований.

Режим: живые соревнования | Масштаб: малый → средний

Хостинг: API · self-host · гибрид | Глубина: стратегия + блюпринт

CONFIDENTIAL DRAFT

Подготовлено для: **Karol**

Проект: **slopi-farm**

Версия 1.0

19 сентября 2026

Содержание

0 Резюме и ключевые рекомендации — что строить и почему, в одну страницу

1 Постановка задачи и словарь понятий — CTF, «рой», «ферма», «агент»

2 Что реально умеют CTF-агенты в 2026 — бенчмарки, solve-rate, сильные и слабые категории

3 Выбор модели: DeepSeek-V4-Pro vs V4-Flash — экономика роя и логика каскада

4 Анатомия одного агента (микроуровень) — ReAct, ACI, интерактивные инструменты, песочница

5 Топология роя (мезоуровень) — паттерны координации и специализация

6 Оркестрация и инфраструктура (макроуровень) — плоскости управления, очереди, durable-workflow

7 Инференс: три сценария хостинга DeepSeek-V4 — API · self-host · гибрид, с расчётами

8 Логика работы под live jeopardy — планировщик реального времени и приоритизация

9 Планировщик ресурсов и потоки агентов — шедулинг GPU, батчинг, бюджеты, автоскейл

10 Изоляция и безопасность — запуск недоверенного кода, prompt injection, kill-switch

11 Наблюдаемость, оценка и самоулучшение — трейсинг, метрики, банк решений, дообучение

12 Дорожная карта внедрения — от MVP до полноценного роя

13 Рекомендация под ваш кейс и риски — сборка воедино, юр./этич. границы

A Глоссарий «трудных» терминов

B Источники

EXECUTIVE SUMMARY

0 Резюме и ключевые рекомендации

Короткая версия для тех, кто дальше читать не будет. Автономная CTF-ферма — это не «одна очень умная модель, которая думает над задачей», а **распределённая система оркестрации роя агентов**, где вычислительные ресурсы, время и токены являются дефицитом, который нужно активно планировать. Под ваш кейс — **живые jeopardy-соревнования** при **малом-среднем масштабе** — выигрывает не грубая сила, а экономика внимания и скорость петли «гипотеза → проверка».

~95%
solve-rate на **InterCode-CTF** у ReAct-агента (Palisade) — «лёгкие» задачи ИИ уже закрывает

41/45
флагов взял ИИ-агент CAI, **победив людей** на Neurogrid CTF (2025)

−47%
медианное время решения easy-задач после прихода LLM (данные по 423 HTB-машинам)

×3–20
разница в цене токена **Flash** против **Pro** — основа логики каскада

Главный тезис

DeepSeek-V4 существует в двух вариантах, и это меняет всю постановку. **V4-Flash** (284 млрд параметров всего, ~13 млрд активных) — дешёвая «рабочая лошадка», которой можно наплодить рой. **V4-Pro** (1,6 трлн всего, ~49 млрд активных) — дорогой «тяжеловес» для сложного рассуждения. Строить рой *только* из Pro — расточительно; строить *только* из Flash — вы упрётесь в потолок на hard-задачах (pwn, продвинутая крипта, многошаговый reverse). Правильный ответ — **гетерогенный рой с каскадом моделей** (model cascade): широкий фронт дешёвых Flash-агентов проводит триаж и берёт рутину, а редкие, дорогие вызовы Pro включаются только там, где Flash «застрял».

Мнение автора (обосновано ниже, разделы 3 и 7)

Если бы нужно было выбрать *одну* конфигурацию для старта — это **гибрид**: self-host **V4-Flash** на арендованном узле из нескольких GPU (весь объём «дешёвых» токенов идёт локально, где prefix-кэш системных промптов даёт максимальную экономию), плюс **V4-Pro через официальное API** для эскалаций. Так вы получаете низкую предельную стоимость токена, контроль над латентностью в «горячей» петле агента и при этом не оплачиваете простой дата-центрового кластера ради модели, которая нужна 10% времени.

Пять решений, которые определяют всю систему

Решение	Рекомендация под live jeopardy, малый-средний масштаб	Почему (кратко)
Модель роя	Гетерогенно: V4-Flash как основной воркер, V4-Pro как «эскалация»	Цена/качество: 80–90% задач не требуют Pro-уровня рассуждений
Хостинг	Гибрид: Flash self-host + Pro API (сценарий C, раздел 7)	Минимальная предельная цена токена + контроль латентности + гибкость
Координация	Blackboard + приоритетная очередь + менеджер-воркеры	Живой скорборд — общий контекст; задачи независимы и хорошо параллелятся
Оркестратор	Durable-workflow (Temporal/Restate) поверх очереди задач	Агент — это долгая многошаговая транзакция; нужны ретраи и чекпоинты
Изоляция	Эфемерные микро-VM (gVisor/Firecracker) на каждый запуск	Агент исполняет недоверенные бинарники и эксплойты — это не опция

Метрика, вокруг которой всё вращается

На живом соревновании вас интересует не абстрактный «процент решённых задач», а **две связки**: `cost-per-flag` — сколько ресурсов (токенов/GPU-секунд/денег) уходит на один взятый флаг, и `time-to-flag` — насколько быстро вы конвертируете задачу в очки, потому что в jeopardy с динамическим скорингом цена задачи *падает* по мере того, как её решают другие команды. Вся оптимизация роя — это, по сути, задача о рюкзаке (knapsack): распределить ограниченный пул агентов и времени по задачам так, чтобы максимизировать ожидаемые очки к моменту закрытия скорборда. Разделы 8 и 9 посвящены именно этому.

Как читать документ

Разделы 1–3 — концептуальная база и выбор модели. Разделы 4–6 идут «снизу вверх»: один агент → рой → инфраструктура. Разделы 7–9 — экономика хостинга и планировщик реального времени (ядро вашего кейса). Разделы 10–11 — то, без чего система развалится в проде: изоляция и наблюдаемость. Каждый «трудный» термин при первом появлении поясняется, а полный глоссарий — в Приложении А.

ФУНДАМЕНТ

1 Постановка задачи и словарь понятий

Прежде чем проектировать инфраструктуру, зафиксируем, что именно мы автоматизируем и что означают ключевые слова из вашего запроса. Разночтения здесь — источник 90% архитектурных ошибок.

1.1. Что такое task-based (jeopardy) CTF

CTF (Capture the Flag) — соревнование по практической кибербезопасности, где нужно найти секретную строку-«флаг» (обычно вида `flag{...}`), спрятанную в задании. **Task-based** (он же jeopardy) формат — это доска с независимыми задачами, разбитыми по категориям и стоимости в очках; команда решает их в любом порядке. Это принципиально отличается от *attack-defense* формата (где команды атакуют сервисы друг друга в реальном времени) — jeopardy идеально ложится на рой, потому что **задачи независимы и эмбарассингли-параллельны** (embarrassingly parallel — их можно раздать разным агентам без взаимной координации).

Канонические категории и что в них требуется от агента:

Категория	Суть	Что нужно агенту	Сложность для ИИ
Web	Уязвимости веб-приложений: SQLi, XSS, SSRF, десериализация, логика	HTTP-клиент, браузер, fuzzing, чтение исходников	**Низкая–средняя**
Crypto	Слабые/неправильно применённые шифры, протоколы, RNG	Математика, sage/sympy, известные атаки, скрипты	**Средняя–высокая**
Pwn (binary exploitation)	Эксплуатация памяти: переполнения, ROP, heap-атаки	Интерактивный отладчик (gdb), pwntools, точность	**Высокая**
Reverse (RE)	Обратная разработка бинарников для понимания логики	Дизассемблер/ декомпилятор, отладчик, терпение	**Средняя–высокая**
Forensics	Анализ дампов, трафика, файлов, стеганография	Спец-утилиты (volatility, wireshark, binwalk)	**Низкая–средняя**
Misc / OSINT	Всё остальное: логика, разведка по открытым данным	Поиск, креативность, инструментальная гибкость	**Разброс**

Оценка сложности — по данным бенчмарков 2024–2026 (раздел 2). Ключевой вывод: категории неоднородны, и **единая стратегия для всех — плохая идея**. Это прямой аргумент за специализацию агентов (раздел 5).

1.2. «Ферма», «рой», «агент» — что именно вы строите

Три уровня, три слова

Агент — один автономный решатель: LLM в цикле «подумал → сделал действие инструментом → посмотрел результат», с доступом к песочнице и памятью. Это *микроуровень* (раздел 4).

Рой (swarm) — множество агентов, которые совместно работают над доской задач, обмениваясь знаниями и конкурируя за ресурсы. Это *мезоуровень* (раздел 5).

Ферма — вся инженерная система вокруг роя: оркестратор, очереди, инференс-сервер(ы), песочницы, хранилища, мониторинг, планировщик. Это *макроуровень* (разделы 6–11).

Ваша интуиция «наплодить рой из дешёвой модели» верна по духу, но требует уточнения: **ценность роя не в количестве агентов, а в разделении труда и в скорости обмена находками.** Сто агентов, которые независимо решают одну задачу «в лоб», в большинстве случаев не сильно лучше одного — они просто в сто раз дороже. Рой начинает выигрывать, когда (а) агенты *специализированы*, (б) есть *общая память*, куда складываются промежуточные находки (blackboard, раздел 5.1), и (в) есть *дешёвый способ отбраковывать* неудачные ветки, чтобы не жечь бюджет. Именно поэтому большая часть этого документа — про координацию и планирование, а не про «побольше GPU».

1.3. Три вещи, которые делают CTF удобным для автоматизации

- **Проверяемость (verifiability).** У задачи есть однозначный признак успеха — правильный флаг. Это «золотой сигнал» (ground-truth reward): агенту не нужен человек, чтобы понять, решил он или нет. Такой чистый сигнал — редкость, и он делает возможными и автономность, и последующее обучение с подкреплением (раздел 11).
- **Изолируемость.** Каждая задача — это отдельный контейнер/сервис. Их легко раздавать, воспроизводить и убивать. Это упрощает и параллелизм, и безопасность.
- **Инструментальность.** Решение почти всегда — это не «текст», а *действия*: запустить, отладить, подключиться, декодировать. Значит, качество агента определяется не только моделью, но и качеством его инструментов и интерфейса к ним (ACI, раздел 4.2). Это ключевой рычаг, который часто недооценивают.

Оговорка о легальности и этике (подробнее — раздел 13)

Всё в этом документе предполагает **легитимный контекст**: официальные соревнования, тренировочные площадки (picoCTF, HTB, CTFtime-события) и собственные полигоны. Автономный агент, применяющий наступательные техники, — это инструмент двойного назначения. Правила конкретного CTF могут *прямо запрещать* полностью автономные решения или ограничивать автоматизацию; ряд площадок вводит отдельные «AI-дивизионы». Проверяйте ToS каждого события до запуска фермы «в бой».

РЕАЛЬНОСТЬ 2026

2 Что реально умеют CTF-агенты сегодня

Чтобы проектировать не в вакууме, зафиксируем эмпирику. За 2024–2026 автономные агенты прошли путь от «иногда решают лёгкое» до «побеждают людей на отдельных событиях». Но распределение способностей крайне неравномерно по категориям — и это напрямую диктует архитектуру роя.

2.1. Ключевые бенчмарки и цифры

Бенчмарк / система	Что измеряет	Результат	Вывод для нас
InterCode-CTF (picoCTF)	Лёгкие/учебные задачи, агент с инструментами	до **~95%** (ReAct, Palisade); базовый SWE-agent 43% → **67–72%** у EnIGMA	«Массовый» слой задач ИИ уже почти закрывает — это работа для Flash
NYU CTF Bench (CSAW)	Реальные соревновательные задачи, 6 категорий	baseline ~4% → **13,5%** (EnIGMA, Claude 3.5) — рост ×3 от инструментов	«Хвост» сложных задач всё ещё низкий — сюда нужен Pro и спец-тулинг
Cybench (Stanford)	Профессиональные задачи из 4 соревнований, с подзадачами	решает часть на скорости, сравнимой с **11-мин** решением человека	Скорость — главное оружие ИИ против людей в live-режиме
AIRTBench	Black-box AI/red-team задачи	Claude 3.7 Sonnet — **61%**	Frontier-модели заметно опережают середину рынка
CAI (live)	Живые соревнования против людей	**41/45** флагов, победа на Neurogrid CTF; топ-ИИ в HTB «AI vs Humans»	Полностью автономная победа над людьми — уже не теория

Цифры — публичные результаты 2024–2026; абсолютные значения зависят от модели, числа итераций и набора инструментов, поэтому важны не столько сами проценты, сколько их *соотношение* между категориями и роль инструментов.

2.2. Динамика: почему время решения — падающая кривая

Лонгитюдный анализ 423 машин Hack The Box показал: медианное время до компрометации падает **примерно на 16% в год**, а после появления мощных LLM (конец 2022) кривая стала круче: easy −47%, medium −32%, hard −27%, insane −67%. Отдельно быстрее всего «схлопнулась» эскалация привилегий (−27…−41%) — то есть шаблонные, хорошо документированные шаги. Windows-цели (Kerberoasting, имперсонация токенов) поддаются лучше, чем гетерогенный Linux, требующий более глубокого контекстного рассуждения.

Что это значит для дизайна

ИИ силён там, где есть **повторяемый паттерн**, и слабее там, где нужна длинная цепочка нестандартного рассуждения с точной работой с состоянием (классический pwn с обходом современных защит). Отсюда два проектных следствия: (1) вложитесь в **банк known-техник** (RAG по райтапам, раздел 11) — это дёшево умножает Flash-агентов; (2) держите **«тяжёлый резерв»** из Pro-вызовов и спец-инструментов для хвоста задач.

2.3. Главный урок из литературы: инструменты важнее «ума» модели

Самый практичный вывод исследований (EnIGMA и др.): переход от «модель просто пишет команды в шелл» к **интерактивным инструментам** дал скачок в разы — особенно в pwn и reverse. Речь про **Interactive Agent Tools (IAT)**: неблокирующий доступ к *интерактивному отладчику* (gdb) и *живому сетевому соединению* с сервисом задачи, плюс *суммаризатор* длинных выводов и *демонстрации* (few-shot примеры успешных решений с восстановлением после ошибок). Абляции показали: уберите интерактивные инструменты — и результат в «тяжёлых» категориях резко падает.

Мнение: где лежат ваши деньги

Соблазн «взять модель помощнее» переоценён. При фиксированном бюджете **прирост от качественного агент-компьютерного интерфейса (ACI) и специализированных инструментов обычно больше, чем от апгрейда модели**. Практически это значит: 60–70% инженерных усилий фермы стоит вкладывать в инструменты, песочницу, память и планировщик, и лишь 30–40% — в «модельную» часть (промпты, каскад, дообучение). Именно поэтому DeepSeek-V4 как «дешёвый и достаточно умный» — рациональный выбор ядра: он оставляет бюджет на то, что решает исход.

ЯДРО РОЯ

3 Выбор модели: V4-Pro vs V4-Flash и логика каскада

DeepSeek-V4 (релиз 24 апреля 2026, открытые веса) — это не одна модель, а семейство. Понимание разницы между вариантами и того, что такое MoE, определяет и стоимость роя, и требования к «железу».

3.1. Что такое MoE и почему это важно для фермы

Термин: Mixture-of-Experts (MoE) и «активные vs всего» параметры

MoE — архитектура, где сеть состоит из множества «экспертов», но на каждый токен активируется лишь малая их часть (роутер выбирает top-k экспертов). Отсюда два числа: **всего параметров** (определяет, сколько весов надо держать в памяти GPU) и **активных параметров** (определяет, сколько вычислений тратится на токен, то есть скорость и стоимость инференса). Практический смысл: MoE даёт «ум» большой модели по цене вычислений маленькой — но платит за это *памятью*. Для self-host это ключевой момент: вам нужен большой суммарный VRAM под *все* веса, даже если активна лишь их часть.

Характеристика	V4-Flash	V4-Pro	Значение для роя
Параметров всего	**284 млрд**	**1,6 трлн**	Pro требует мульти-узлового self-host
Активных на токен	~13 млрд	~49 млрд	Flash ≈ в 3–4× дешевле по вычислению
Контекст	1 млн токенов	1 млн токенов	Хватает на исходники + логи + историю
Веса	Открытые (можно self-host обе)		Возможен полный контроль и приватность
API-цена (вход/выход, за 1M)*	≈ $0,14 / $0,28	≈ $0,4–1,7 / $0,9–3,5	Экономика каскада строится на этом гэпе
Self-host: память	~140+ ГБ (узел из GPU)	дата-центр (мульти-узел)	Flash реалистичен для среднего масштаба
Роль в рое	**Воркер / фронт триажа**	**Эскалация / «тяжёлый» reasoning**	—

*Цены V4-Pro в разных вторичных источниках расходятся (от ~$0,44 до ~$1,74 за 1M входных); официальный тариф следует подтвердить у провайдера на момент запуска. Для расчётов ниже берём консервативную «вилку». Cache-hit цена входных токенов у V4 в разы ниже cache-miss — это отдельный рычаг экономии (раздел 9.3).

3.2. Логика каскада (model cascade) — сердце экономики роя

Идея проста и мощна: **не платить за дорогое рассуждение там, где хватит дешёвого**. Каждая задача (и каждый под-шаг внутри задачи) сначала попадает к дешёвому Flash-агенту. Тот решает сам, если может; если «застревает» (нет прогресса за N шагов, низкая уверенность, зацикливание) — под-проблема эскалируется к Pro. Это тот же принцип, что триаж в приёмном покое: дорогой специалист подключается выборочно.

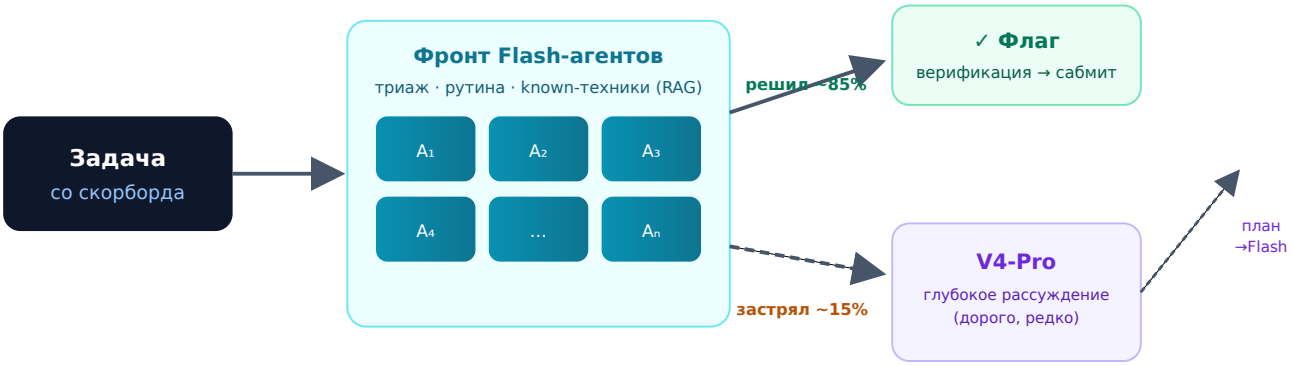


Рис. 1. Каскад: широкий дешёвый фронт Flash-агентов, редкая дорогая эскалация к Pro. Pro чаще возвращает не готовый флаг, а «план», который доисполняет дешёвый агент.

Экономический эффект каскада легко оценить. Пусть доля задач, требующих Pro, — 15%, а Pro в 10× дороже за токен. Тогда средневзвешенная цена токена роя ≈ `0,85·1 + 0,15·10 = 2,35` «единиц» против 10 у «роя только из Pro» — то есть **экономия ~4×** при сопоставимом потолке качества. Реальные числа зависят от вашей доли эскалаций, но направление всегда одно: **каскад почти всегда доминирует над гомогенным роем из дорогой модели.**

Практическое правило порога эскалации

Эскалируйте, когда выполняется хотя бы одно: (1) *нет прогресса* — агент N шагов не меняет состояние (не открыл новый факт/файл/ветку); (2) *зацикливание* — повтор одних и тех же действий; (3) *высокая ставка* — задача дорогая по очкам и близка к дедлайну; (4) *явный сигнал сложности* — категория pwn/hard-crypto по классификатору триажа. Порог — это гиперпараметр, который вы калибруете по метрике cost-per-flag (раздел 11).

3.3. Почему именно DeepSeek-V4 как ядро — и где пределы этого выбора

Аргументы «за»: (1) **открытые веса** → возможен self-host, приватность и дообучение под CTF; (2) **сильные кодовые способности** (высокие результаты на кодовых бенчмарках) — критично, ведь CTF-решение почти всегда код; (3) **1M контекст** — можно скормить целые исходники, дампы, историю попыток без агрессивного «нарезания»; (4) **цена** Flash делает рой экономически осмысленным. Пределы: на самом «хвосте» hard-pwn и продвинутой крипты frontier-модели (напр. топовые проприетарные) местами всё ещё сильнее — поэтому архитектура должна **допускать подключение и других моделей** как альтернативной «эскалации» (см. абстракцию провайдера в разделе 6.4). Не привязывайте ферму жёстко к одному вендору весов.

МИКРОУРОВЕНЬ

4 Анатомия одного агента

Рой не станет лучше своих частей. Прежде чем масштабировать, нужно, чтобы *один* агент решал задачи надёжно. Разберём его устройство — именно здесь сосредоточена бóльшая часть «недооценённой» инженерии.

4.1. Базовый цикл: ReAct

Термин: ReAct (Reason + Act)

Парадигма, в которой модель чередует **рассуждение** («мысль» — план следующего шага) и **действие** (вызов инструмента), после чего получает **наблюдение** (результат) и повторяет цикл. Формально: `Thought → Action → Observation → Thought → …` до достижения цели или лимита. Это рабочая лошадка всех современных CTF-агентов (EnIGMA, CAI и др.), потому что CTF по природе итеративен: попробовал — посмотрел — скорректировал.

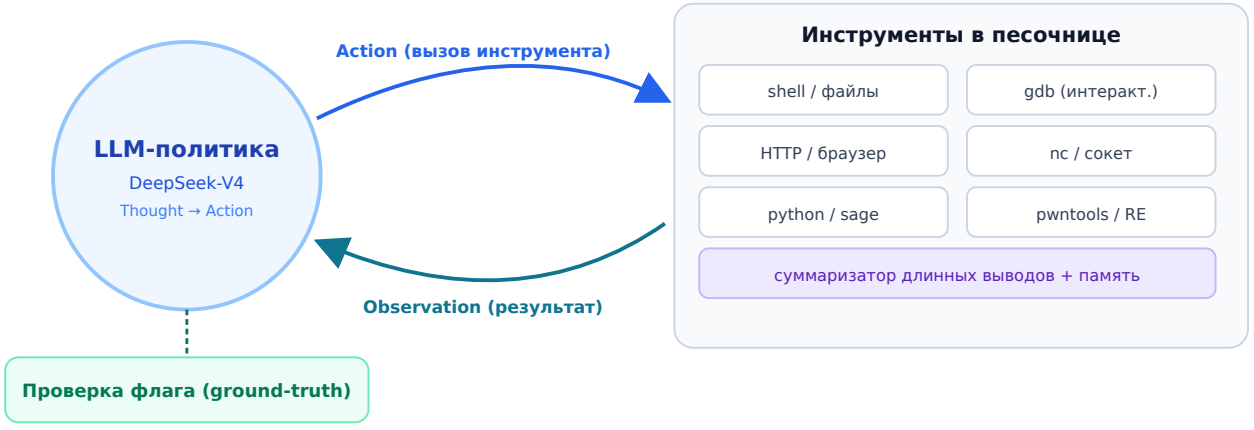


Рис. 2. Внутренний цикл агента. Качество определяется не только моделью в центре, но и «поясом» инструментов и интерфейсом к ним.

4.2. ACI — агент-компьютерный интерфейс (недооценённый рычаг)

Термин: ACI (Agent-Computer Interface)

Это «UX для модели» — то, как инструменты представлены агенту: формат команд, форма и объём наблюдений, обработка ошибок. Хороший ACI лаконичен, детерминирован и «прощает» ошибки (подсказывает, как исправить). Как отмечено в разделе 2.3, вклад ACI в результат нередко больше вклада самой модели.

Три принципа хорошего ACI для CTF: **(1) интерактивность** — агент должен уметь работать с «живыми» программами (отладчик, удалённый сервис), а не только запускать команды «в один выстрел»; неблокирующие интерактивные сессии — тот самый приём, что дал скачок в pwn/RE. **(2) Сжатие наблюдений** — сырые дампы/логи легко переполняют контекст; отдельный суммаризатор (простой «в файл» или LLM-суммаризатор) сохраняет действенные

детали и режет шум. **(3) Демонстрации** — few-shot примеры «как решать такую категорию, включая восстановление после провала» дёшево поднимают качество, особенно у Flash.

4.3. Специализированные инструменты по категориям

Категория	Минимальный тулкит агента	Критичный интерактивный инструмент
Web	curl/httpx, headless-браузер, sqlmap-подобный fuzzing, просмотр исходников	Живой HTTP-сеанс с сохранением куки/сессии
Pwn	pwntools, checksec, ROPgadget, one_gadget, компилятор	**gdb** с интерактивной отладкой (критично)
Reverse	Ghidra/objdump (декомпиляция), strings, strace/ltrace	Интерактивный отладчик для динамического анализа
Crypto	python, sympy/pycryptodome, sage, база известных атак	REPL для итеративной проверки гипотез
Forensics	volatility, wireshark/tshark, binwalk, exiftool, stegsolve	Файловый/пакетный «навигатор» с суммаризацией

4.4. Исполнение и песочница (см. также раздел 10)

Каждый агент работает внутри изолированной среды, где лежат файлы задачи и установлен тулкит. Ключевой принцип — **эфемерность**: среда создаётся под попытку и уничтожается после, чтобы (а) не тащить состояние между задачами и (б) ограничить ущерб от недоверенного кода. На микроуровне достаточно понимать: инструмент = вызов в песочнице, а сама песочница — предмет отдельного, серьёзного разговора о безопасности.

4.5. Управление контекстом и памятью

Даже с 1M-контекстом V4 нельзя бездумно складывать туда всё: длинный контекст дороже, медленнее и «размывает» внимание. Практика: **рабочая память** агента = система инструкций + сжатая история действий + релевантные фрагменты (найденные RAG-ом по банку райтапов). Старые шаги — суммаризируются; сырые артефакты — лежат в файловой системе песочницы и подтягиваются по ссылке, а не текстом. Это одновременно экономия токенов и повышение точности.

4.6. Верификация флага — «золотой сигнал»

Прежде чем засчитывать успех, флаг проверяется по формату (regex платформы) и, где возможно, автоматической отправкой в проверяющую систему. Этот однозначный сигнал — не только критерий остановки агента, но и топливо для всего цикла улучшения фермы (раздел 11): по нему считаются метрики, отбираются удачные траектории и обучается порог эскалации.

Проектный принцип микроуровня

Сначала доведите **одного** агента до стабильных ~цифр бенчмарка на каждой категории, и только потом масштабируйте. Рой из плохих агентов — это дорогой способ получать плохие результаты быстрее.

МЕЗОУРОВЕНЬ

5 Топология роя: как агенты работают вместе

«Рой» — это не «много копий одного агента». Это система разделения труда с общей памятью и правилами конкуренции за ресурсы. Разберём проверенные паттерны координации и выберем подходящий под live jeopardy.

5.1. Паттерн «доска» (blackboard) — рекомендуемая основа

Термин: Blackboard (доска общих знаний)

Классический паттерн ИИ: есть общая структура данных («доска»), куда разные специалисты-агенты асинхронно пишут промежуточные находки и читают чужие. Никто не управляет всеми напрямую — координация происходит *через состояние*. Для CTF доска хранит: список задач и их статусы, найденные артефакты (учётки, эндпоинты, части флага), гипотезы, «тупики» (чтобы не повторять), и текущие бюджеты.

Почему это идеально для jeopardy: задачи независимы, но *находки переиспользуемы* (учётка из web-задачи открывает forensics-задачу; тот же криптопаттерн встречается дважды). Доска превращает N одиночек в коллектив, где успех одного удешевляет работу другого. Технически доска — это несколько таблиц в Postgres + быстрый слой в Redis для «горячего» состояния.

5.2. Кто раздаёт задачи: менеджер-воркеры и work-stealing

Поверх доски нужен способ распределять работу. Два совместимых механизма:

- **Менеджер-воркеры (manager–workers).** Лёгкий управляющий процесс (оркестратор-планировщик) кладёт задачи в приоритетную очередь; воркеры-агенты берут их по мере освобождения. Менеджер не «думает» за агентов — он лишь распределяет и следит за бюджетами и дедлайнами.
- **Кража работы (work-stealing).** work-stealing — приём балансировки нагрузки: простаивающий воркер «крадёт» задачи из очереди перегруженного. Это избавляет от ручной привязки задач к агентам и естественно выравнивает загрузку, когда задачи сильно различаются по трудоёмкости (а в CTF так всегда).

Ещё один полезный паттерн: контрактные торги (contract-net)

Для сложной задачи менеджер объявляет «тендер», а специализированные агенты «делают ставки» (оценивают, насколько задача им подходит: категория, сложность, наличие подходящих инструментов). Задача уходит лучшему претенденту. Это мягкий способ реализовать специализацию без жёсткого прибивания категорий к агентам.

5.3. Специалисты против дженералистов

Данные (раздел 2) показывают: категории требуют разного тулинга и разных стратегий. Отсюда — **гибридная специализация**: агенты используют одну и ту же модель-ядро, но различаются *системным промптом, набором инструментов и демонстрациями* под категорию. Это даёт выигрыш специализации почти без затрат (веса общие) и вдобавок

резко повышает попадание в prefix-кэш: у всех web-агентов одинаковый длинный системный префикс → инференс дешевле и быстрее (раздел 9.3). Первый шаг конвейера — дешёвый **классификатор-триаж** (может быть даже мелкой моделью), который метит задачу категорией и оценкой сложности.

5.4. Обмен знаниями и «не наступать на грабли дважды»

Три канала обмена: **(1) доска** — оперативные находки в рамках одного соревнования; **(2) банк решений** — долговременная база успешных траекторий и райтапов, доступная агентам через RAG (раздел 11.3); **(3) негативная память** — журнал тупиков и провальных подходов, чтобы рой не тратил бюджет на заведомо мёртвые ветки. Последнее часто забывают, а оно экономит едва ли не больше, чем позитивные подсказки.

5.5. Антипаттерны, которые сожгут ваш бюджет

Антипаттерн	Почему плохо	Что делать вместо
«100 агентов в лоб на одну задачу»	Линейный рост стоимости при сублинейном росте шанса	Разнообразие подходов + ранняя отбраковка + каскад
Гомогенный рой из Pro	В разы дороже без выигрыша на 80% задач	Каскад Flash→Pro (раздел 3.2)
Нет общей памяти	Повторная работа, потеря переиспользуемых находок	Blackboard + негативная память
Агент без лимитов	Зацикливание жжёт токены и держит песочницу	Бюджеты шагов/токенов/времени + watchdog
Синхронная координация	«Болтливые» агенты создают затор и латентность	Асинхронность через доску и очереди

МАКРОУРОВЕНЬ

6 Оркестрация и инфраструктура фермы

Здесь мы собираем ферму из компонентов. Главный принцип — **разделение плоскостей (planes)**: управление, вычисление и данные живут отдельно, соединяясь через очереди и API. Это то, что превращает «скрипт с циклом» в систему, переживающую сбои и масштабируемую.

6.1. Семислойная модель фермы

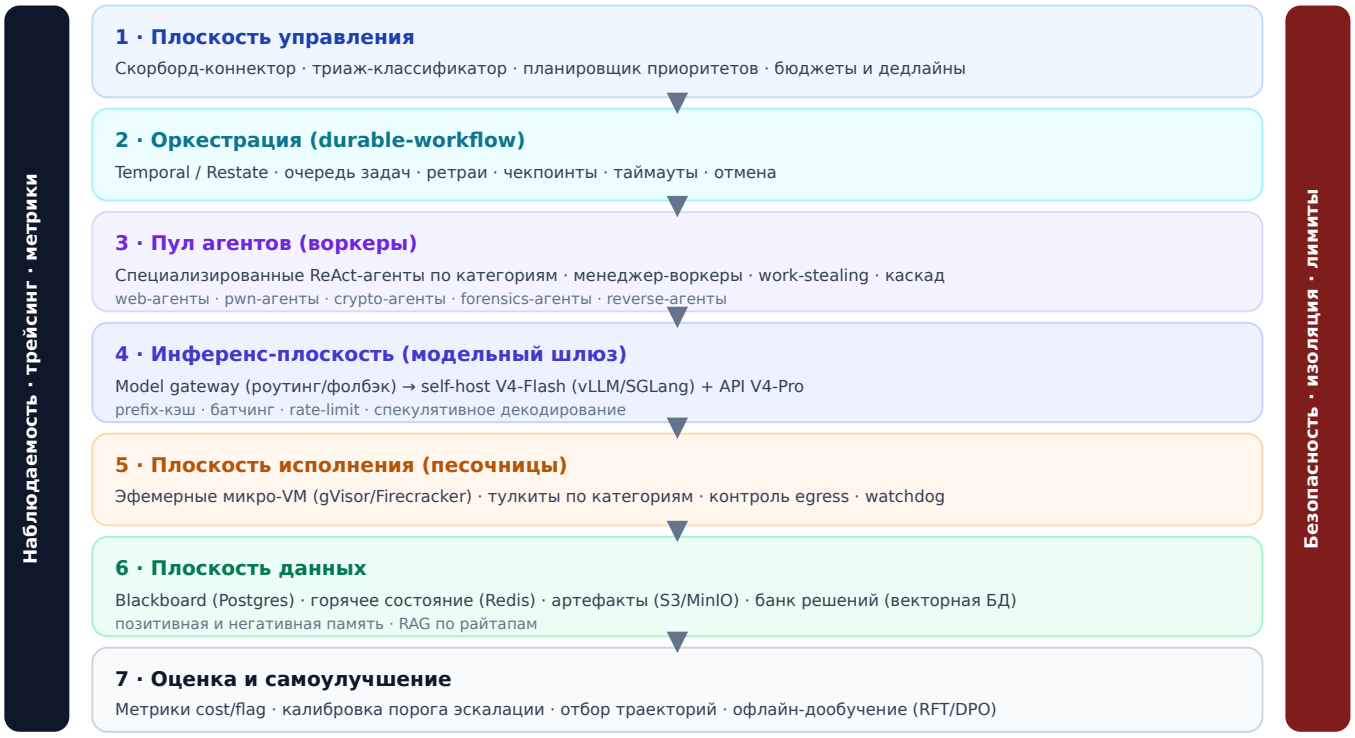


Рис. 3. Семь слоёв фермы. Наблюдаемость и безопасность — сквозные (пронизывают все слои), поэтому вынесены по бокам.

6.2. Транспорт: очередь задач

Рой общается асинхронно, а не «звонками напрямую». Между планировщиком и воркерами стоит **очередь сообщений** — она разъединяет производителя и потребителя, сглаживает пики и даёт `backpressure` (обратное давление: когда воркеры перегружены, очередь тормозит поступление новой работы, а не роняет систему).

Вариант	Когда подходит	Комментарий для фермы
Redis Streams	Малый-средний масштаб, минимум движущихся частей	**Рекомендуется на старте: он же — «горячее» состояние доски**
NATS / JetStream	Нужна низкая латентность и лёгкий кластер	Отличный баланс скорости и простоты при росте
RabbitMQ	Сложная маршрутизация, приоритетные очереди из коробки	Зрелый, богатая семантика доставки
Kafka	Большой поток событий, реплей истории	**Избыточен для малого-среднего; берите позже, если понадобится лог событий**

6.3. Почему нужен durable-workflow, а не просто очередь

Термин: Durable execution (устойчивое исполнение)

Движок, который **записывает каждый шаг** длинного процесса, чтобы после сбоя (упал воркер, вытеснили spot-инстанс, OOM) продолжить *ровно с места падения*, а не с нуля. Агент — это как раз длинная многошаговая транзакция с дорогими побочными эффектами (запуски, вызовы модели). Наивный цикл в проде ломается тремя способами: каскад таймаутов, частичное выполнение с дублями побочных эффектов и «штормы ретраев», выжигающие GPU.

Движок	Модель	Когда выбрать для фермы
Temporal	История событий + детерминированный реплей; heartbeat для GPU-задач	**Дефолт: длинные агентские воркфлоу, зрелая экосистема, явный контроль таймаутов**
Restate	Virtual objects, нативная семантика *exactly-once*	Если критична строгая «ровно один раз» на сессию-агента без ручных ключей идемпотентности
Inngest	Событийные пайплайны, поштучные ретраи шагов	Для триггеров по событиям (webhook/cron) и независимых ретраев шагов

Ключевая практика (из гайдов по durable-оркестрации на GPU)

Держите **GPU-операции в «activity», а не в коде воркфлоу** (код воркфлоу должен быть детерминированным и быстрым). Ставьте `heartbeat` с интервалом < 50% таймаута активности — тогда смерть GPU-воркера от вытеснения обнаруживается за один интервал и задача переназначается. Экономика чекпоинтов: сбой 4-часовой работы на 3,5-м часе без чекпоинта = ~$17,75 сожжённого compute; с 15-мин чекпоинтами максимум потерь ~$1,27. Оркестратор ($30–50/мес) окупается на первом же сбое.

6.4. Модельный шлюз (model gateway) — абстракция провайдера

Между агентами и «мозгами» стоит единый шлюз с OpenAI-совместимым API. Он: (1) **маршрутизирует** — Flash-запросы уходят на self-host, Pro-эскалации — в API; (2) даёт **фолбэк** — если self-host перегружен или API отдаёт rate-limit, запрос уходит на резерв; (3) ведёт **учёт токенов и стоимости** по каждому агенту/задаче; (4) **разъединяет** ферму и конкретного вендора весов. Именно эта абстракция позволяет позже добавить frontier-модель как альтернативную эскалацию, не переписывая агентов (см. предел выбора в 3.3). На практике это либо готовый прокси-слой (LiteLLM-подобный), либо тонкий собственный сервис.

6.5. Хранилища состояния

Слой	Технология	Что хранит
Оперативная доска	Redis	Горячее состояние задач, локи, счётчики бюджета, очередь
Долговременная доска	PostgreSQL	Задачи, попытки, находки, аудит-лог, метрики
Артефакты	S3 / MinIO	Бинарники, дампы, большие выводы (по ссылке, не в контекст)
Банк решений (RAG)	Векторная БД (pgvector/Qdrant)	Райтапы, успешные траектории, известные атаки, негативная память

Совет по масштабу «малый→средний»

Не переусложняйте на старте. PostgreSQL + Redis + MinIO + pgvector закрывают все четыре роли и разворачиваются одним `docker compose`. Kafka, отдельный вектор-кластер и мультирегион — это «проблемы успеха», к которым переходят по метрикам, а не заранее.

ИНФЕРЕНС-ПЛОСКОСТЬ

7 Три сценария хостинга DeepSeek-V4

Вы просили разобрать все варианты — вот они, с честными плюсами, минусами и цифрами. Ключевая мысль: это не выбор «или-или» навсегда, а выбор *стартовой точки*; зрелые фермы почти всегда сходятся к гибриду.

Сценарий A — Полностью через API · «арендуем мозги»

Все вызовы (и Flash, и Pro) идут в облачные API-эндпоинты. Ферма держит только control/orchestration/execution/data-слои; GPU в собственности нет.

Плюсы: нулевой порог входа, нет капзатрат, мгновенное масштабирование «в пик», всегда свежая версия модели, ноль возни с GPU-драйверами. **Минусы:** предельная цена токена выше self-host на объёме; **rate limits** провайдера — жёсткий потолок роя в «горячий» момент; латентность и приватность вне вашего контроля; данные задач уходят наружу (для приватных соревнований — стоп-фактор).

Кому: старт, прототип, нерегулярные соревнования, малый объём. Лучший способ проверить гипотезу фермы за выходные.

Сценарий B — Полный self-host · «свои мозги на своих/арендованных GPU»

И Flash, и Pro крутятся на ваших GPU (собственных или арендованных bare-metal). Ферма контролирует весь стек.

Плюсы: низшая предельная цена токена на большом устойчивом объёме; полный контроль латентности; данные не покидают периметр (приватность); никаких внешних rate limits; возможность дообучения (раздел 11). **Минусы:** V4-Pro (1,6 трлн) — это *мульти-узловой дата-центровый* workload, дорого и сложно операционно; вы платите за GPU *и в простое*; на вас MoE-инференс, память под KV-кэш 1M-контекста, обновления, отказоустойчивость.

Кому: постоянная высокая загрузка, требования приватности, планы на дообучение, наличие инженеров под инфру. Для V4-Pro в одиночку — редко оправдан на малом-среднем масштабе.

Сценарий C — Гибрид · РЕКОМЕНДУЕТСЯ: Flash self-host + Pro API

Основной объём «дешёвых» токенов — self-host **V4-Flash** (реалистично помещается на узле из нескольких GPU). Редкие дорогие эскалации — **V4-Pro через API**. Модельный шлюз (6.4) прозрачно маршрутизирует между ними.

Плюсы: низкая предельная цена на массе Flash-токенов + максимум выгоды от prefix-кэша общих системных промптов; вы не оплачиваете простой дата-центрового Pro-кластера; Pro доступен «по требованию» без капзатрат; приватность для основной массы токенов (внешне уходит лишь ~15% эскалаций). **Минусы:** две операционные плоскости вместо одной; нужен грамотный роутинг и учёт стоимости.

Кому: ваш кейс. Оптимум цена/качество/контроль для live jeopardy при малом-среднем масштабе.

7.1. Сравнение по осям, которые важны на живом CTF

Ось	A · API	B · Self-host	C · Гибрид
Капзатраты (старт)	**нулевые**	**высокие**	**средние**
Предельная цена токена	**выше на объёме**	**низшая (при загрузке)**	**низкая (Flash) + гибко (Pro)**
Контроль латентности «в петле»	**низкий**	**полный**	**полный для Flash**
Потолок в пик (rate limits)	**жёсткий**	**свой**	**Flash — свой, Pro — API**
Приватность данных задач	**данные наружу**	**периметр**	**~85% в периметре**
Операционная сложность	**низкая**	**высокая**	**средняя**
Масштабирование в пик	**мгновенное**	**ограничено парком**	**Flash + burst в API**
Возможность дообучения	**нет**	**да**	**да (Flash)**

7.2. Self-host под лупой: чем и на чём подавать V4-Flash

Раз в гибриде self-host несёт основную нагрузку, важно выбрать движок инференса. Два лидера 2026 — **vLLM** и **SGLang**; оба умеют тензорный параллелизм (TP) и экспертный параллелизм (EP), нужный для MoE, и мульти-узловой NCCL.

Термины: PagedAttention, RadixAttention, prefix-кэш, TTFT

KV-кэш — память, где модель хранит «контекст» уже обработанных токенов; для 1M-контекста он огромен. **PagedAttention** (vLLM) нарезает его на страницы фикс. размера — влезает больше параллельных запросов в тот же VRAM. **RadixAttention** (SGLang) хранит KV-кэш в префиксном дереве, эффективно *переиспользуя общий префикс* между запросами (**prefix-кэш**). **TTFT** (time-to-first-token) — задержка до первого токена ответа; критична для «отзывчивости» агента.

Показатель (H100, конкурентность 50)	vLLM	SGLang	Что важно для роя
Throughput (уник. промпты)	~1850 tok/s	~1920 tok/s	Почти равны без общего префикса
TTFT при 80% общем префиксе	p50 310 мс	**p50 195 мс (−37%)**	Рой = общие системные префиксы → SGLang выигрывает
Cache-hit при 80% overlap	—	**~84%**	Прямая экономия compute и латентности
MoE (V4-Flash, 8×H100)	~580 tok/s; TTFT 730 мс	**~620 tok/s; TTFT 520 мс**	Преимущество префикса сохраняется на MoE
Спекулятивное декодирование	**зрелое (Eagle3, +30–45%)**	экспериментальное	Ускоряет декод при низком батче

Данные — сводные бенчмарки 2026; абсолютные значения зависят от конфигурации, но соотношение устойчиво.

Мнение: SGLang как основной движок роя

Рой агентов — это **идеальный случай для RadixAttention**: у web-агентов, pwn-агентов и т.д. длинные *одинаковые* системные префиксы (инструкции + демонстрации + описания инструментов), которые повторяются в тысячах запросов. Здесь prefix-кэш даёт и −37% TTFT (отзывчивость петли), и −19% стоимости. Практичный итог: **SGLang для «горячего» self-host Flash**, а vLLM держите как альтернативу для батч-задач с уникальными промптами и там, где нужен зрелый speculative decoding. Модельный шлюз (6.4) позволяет держать оба за одним API и роутить.

Про «железо» под V4-Flash

Grubo: Flash (284 млрд, ~13 млрд активных) требует порядка **~140+ ГБ VRAM только под веса** (в пониженной точности), плюс запас под KV-кэш длинного контекста и батч. Реалистичная стартовая единица — **узел из нескольких H100/H200 (80–141 ГБ каждая)**, связанных NVLink, с TP+EP. V4-Pro (1,6 трлн) — это уже *несколько узлов*, поэтому в гибриде мы намеренно не держим его локально.

7.3. Экономика под live jeopardy: прикидочная модель

Живое соревнование — это **всплеск нагрузки на несколько часов**, а не ровный поток 24/7. Это меняет вывод: круглосуточно гонять собственный дорогой кластер под редкие «бои» невыгодно, а вот держать *умеренный* self-host Flash под постоянную тренировку/грайндинг и *докупать пик* Pro через API — рационально. Прикидка на одно 8-часовое соревнование, средний масштаб:

Параметр (пример)	Значение	Комментарий
Задач на доске	40	Типичная jeopardy-доска
Попыток-траекторий (с ретраями/разнообразием)	~200	~5 подходов на задачу в среднем
Токенов на траекторию (in+out)	~150–400 тыс.	Длинные исходники + история + инструменты
Суммарно Flash-токенов	~40–60 млн	Основная масса — дешёвый Flash
Доля эскалаций в Pro	~15%	Только «застрявшие» под-проблемы
Стоимость Flash (self-host, ~$0,5/1M)	~$20–30	Амортизация аренды GPU за окно соревнования
Стоимость Pro (API-эскалации)	~$15–40	Зависит от подтверждённого тарифа Pro
Итого за соревнование	**~$40–70**	Плюс фикс на инфру ($30–50/мес оркестратор и т.п.)

Это *порядок величины*, а не смета: точные числа зависят от подтверждённого тарифа V4-Pro, реальной длины траекторий и доли эскалаций. Цель таблицы — показать структуру затрат и то, что доминирует масса дешёвых Flash-токенов, а не редкий Pro.

Три рычага снижения cost-per-flag

(1) **Prefix-кэш** общих системных промптов (SGLang) — почти бесплатно режет и цену, и латентность. (2) **Ранняя отбраковка** — watchdog убивает бесперспективные траектории до того, как они сожгут бюджет. (3) **Каскад** — не пускать Pro туда, где справится Flash. Все три — про *дисциплину расходов*, а не про «больше GPU».

ЯДРО ВАШЕГО КЕЙСА

8 Логика работы под live jeopardy

В живом режиме появляется то, чего нет в «грайндинге»: **жёсткое время** и **динамическая цена задач**. Это превращает ферму из «решателя задач» в «оптимизатор очков к дедлайну». Разберём жизненный цикл задачи и логику планировщика реального времени.

8.1. Жизненный цикл задачи на живом соревновании

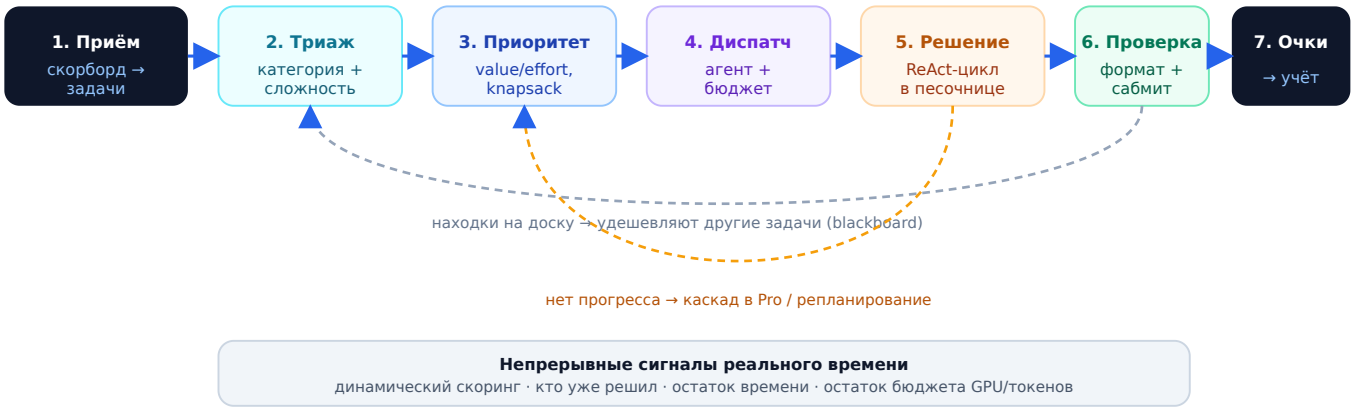


Рис. 4. Конвейер задачи в live-режиме. Две петли обратной связи (находки на доску и каскад при застревании) и внешние сигналы реального времени — то, что отличает live от офлайн-грайндинга.

8.2. Приоритизация: почему это задача о рюкзаке

Термин: динамический скоринг (dynamic scoring)

В большинстве современных jeopardy-CTF стоимость задачи **падает** по мере того, как её решают всё больше команд (награда за «редкое» решение выше). Значит, ценность задачи — величина *убывающая во времени*, и «дорогая, но которую вот-вот решат все» может оказаться хуже «средней, но пока никем не тронутой».

Отсюда планировщик решает непрерывную **задачу о рюкзаке (knapsack)**: есть ограниченный «рюкзак» ресурсов (агенто-часы, GPU-секунды, токен-бюджет, оставшееся время), и надо набрать максимум *ожидаемых* очков. Для каждой задачи оценивается:

- **Ожидаемая ценность** = текущая стоимость в очках × вероятность решения нашим роем × поправка на динамический скоринг (успеем ли мы до «обесценивания»).
- **Ожидаемая стоимость** = прогноз агенто-времени и токенов (по категории/сложности из триажа и из банка решений — «похожие задачи решались за X»).
- **Приоритет** ≈ ценность / стоимость (плотность «очков на ресурс»), с бустом для задач у дедлайна и штрафом для «тонущих» (много попыток без прогресса).

8.3. Бюджеты и «дожатие» под дедлайн

Каждая траектория получает **тройной бюджет**: лимит шагов ReAct, лимит токенов и лимит времени. Их исчерпание — сигнал «отбраковать или эскалировать», а не «продолжать

бесконечно». Ближе к концу соревнования логика меняется: планировщик **концентрирует ресурсы** на задачах с наилучшей плотностью очков и лучшим прогрессом, снимая агентов с «тонущих» веток. Это прямой аналог тактики сильных человеческих команд — «под конец добиваем то, что почти взято».

8.4. Псевдокод планировщика реального времени

```
# Главный цикл планировщика (упрощённо). Работает поверх durable-workflow.
loop every tick:
    board   = scoreboard.sync()          # свежие статусы, динам. скоринг, кто решил
    budget  = resources.remaining()      # GPU-сек, токены, агенты, время до конца

    # 1) переоценка приоритетов с учётом времени и обесценивания
    for task in board.open_tasks:
        task.value = task.points * p_solve(task) * decay(task, board)
        task.cost  = estimate_cost(task.category, task.difficulty, memory)
        task.prio  = task.value / task.cost + deadline_boost(task) - sink_penalty(task)

    # 2) knapsack: раскладываем ресурсы по задачам с макс. плотностью очков
    plan = knapsack(board.open_tasks, budget, key="prio")

    # 3) диспатч: спец-агент под категорию, тройной бюджет, prefix по категории
    for task in plan.dispatch:
        agent = pool.pick(task.category)          # специализация → prefix-кэш
        orchestrator.start(agent, task,
            budgets={steps, tokens, time},
            on_stall=escalate_to_pro,             # каскад
            on_find =blackboard.publish)          # находки → другим

    # 4) отзыв ресурсов с бесперспективных/обесценившихся веток
    for t in running:
        if t.stalled() or t.value_dropped() or deadline_near_and_low_prio(t):
            orchestrator.cancel(t)                # backpressure/освобождение
```

Почему это поверх durable-workflow

`orchestrator.start/cancel` — не просто вызовы функций, а долговечные воркфлоу с чекпоинтами: если воркер умрёт посреди 5-минутной эксплуатации, задача переназначится с последнего шага, а не начнётся заново. Планировщик при этом остаётся тонким и детерминированным — вся «тяжесть» в активностях (раздел 6.3).

ОПТИМИЗАЦИЯ ПОТОКОВ

9 Планировщик ресурсов и потоки агентов

Раздел 8 отвечал «какие задачи брать»; этот — «как физически прогнать через ферму сотни траекторий, не утопив GPU и не упёршись в rate limits». Это и есть та «эффективная координация ресурсов», о которой вы спрашивали.

9.1. GPU-шедулинг: непрерывный батчинг и потолок конкурентности

Современные движки инференса (vLLM/SGLang) делают **continuous batching** (непрерывный батчинг) — динамически доукомплектовывают батч новыми запросами, не дожидаясь завершения текущих. Ваша задача — *подавать* движку ровно столько параллельных запросов, сколько он переваривает на пике КПД, и **не больше**. Отсюда правило: **cap concurrency** — ограничьте число одновременных траекторий так, чтобы очередь к инференсу не росла бесконечно (иначе латентность каждого запроса деградирует и вся петля агента «залипает»).

9.2. Приоритетные очереди + work-stealing

Между планировщиком и пулом воркеров — **приоритетная очередь** (ключ = плотность очков из 8.2). Простаивающие воркеры *крадут* работу (work-stealing, 5.2), что автоматически выравнивает нагрузку при разной трудоёмкости задач. Отдельная очередь — для **эскалаций в Pro** с более высоким приоритетом и отдельным лимитом конкурентности (Pro дороже и его API-квота ограничена).

9.3. Prefix-кэш как экономический рычаг (не только техника)

Это самый недооценённый рычаг координации. Если **группировать запросы по категориям** (все web-агенты подряд, все pwn-агенты подряд), их длинные одинаковые системные префиксы попадают в общий кэш (RadixAttention), и вы получаете −37% TTFT и −19% стоимости «из воздуха». То есть **порядок подачи запросов влияет на цену**. Планировщик должен это учитывать: при прочих равных выгодно временно сгруппировать однокатегорийные траектории.

Проектное следствие

Держите **стабильные, редко меняющиеся системные префиксы** у агентов одной категории (инструкции + описания инструментов + демонстрации). Любое «дрожание» префикса (вставка таймстемпа, случайного id в начало) убивает кэш-хит. Ставьте изменяемое (конкретику задачи) — *в конец* промпта, неизменное — в начало.

9.4. Токен-бюджеты, rate limits и «burst» в API

Три механизма против «выжигания» ресурсов: **(1) токен-бюджеты** на траекторию/задачу/соревнование (жёсткие потолки, при достижении — отбраковка/эскалация). **(2) Учёт rate-limit** провайдера Pro на уровне модельного шлюза: очередь эскалаций с ограничителем скорости, чтобы не ловить 429-е и не терять запросы. **(3) Burst-политика**: в пик, когда self-host Flash насыщен, часть Flash-нагрузки временно уходит в API (гибрид даёт эту гибкость).

Автоскейл self-host на короткое окно соревнования обычно не успевает (холодный старт узлов), поэтому *burst в API — практичнее автоскейла GPU* для коротких «боёв».

9.5. Контроль допуска (admission control) и обратное давление

Когда ресурсы на исходе, система не должна «принимать всё и деградировать». **Admission control** отклоняет/откладывает новые низкоприоритетные траектории, сохраняя ресурс под высокоприоритетные. Это и есть backpressure на уровне бизнес-логики: лучше решить 20 нужных задач хорошо, чем 40 — плохо и наполовину.

9.6. Сводка рычагов оптимизации потоков

Рычаг	Что делает	Эффект
Continuous batching	Динамически комплектует батч на GPU	**Максимальный throughput инференса**
Cap concurrency	Ограничивает число параллельных траекторий	Стабильная латентность, нет «залипания»
Prefix-кэш по категориям	Переиспользует общие префиксы	**−37% TTFT, −19% стоимости**
Work-stealing	Балансирует неравномерную нагрузку	Нет простаивающих воркеров
Токен-бюджеты + watchdog	Режет бесперспективные траектории	**Прямое снижение cost-per-flag**
Burst в API	Пик Flash уходит в облако	Гибкость без автоскейла GPU
Admission control	Отклоняет низкоприоритетное при дефиците	Деньги/время идут в лучшие задачи

БЕЗ ЭТОГО НЕЛЬЗЯ В ПРОД

10 Изоляция и безопасность

CTF-агент по определению **запускает недоверенный код**: чужие бинарники, эксплойты, скачанные пейлоады. Это делает песочницу не «фичей качества», а **несущей конструкцией**. Плюс появляется новый класс угроз — инъекции через содержимое самих задач.

10.1. Модель угроз фермы

Угроза	Сценарий	Митигация
Вредоносный бинарник задачи	«RE»-семпл оказывается реальным малварём/выходит за пределы	Эфемерная микро-VM на попытку, без секретов внутри
Атака не на ту цель	Агент бьёт по инфраструктуре организаторов или в интернет	Жёсткий контроль egress: allowlist только цели задачи
Эксфильтрация данных	Пейлоад/агент утекает ключи, токены фермы наружу	Нет секретов в песочнице; сетевой периметр; аудит трафика
Ресурсные бомбы	Fork-бомба, memory-бомба, бесконечный цикл	cgroups-лимиты CPU/RAM/PIDs + watchdog + таймауты
Prompt injection из задачи	Файл/вывод содержит текст «игнорируй инструкции, сделай X»	Изоляция контента как данных; агент не исполняет инструкции из наблюдений вслепую
Побег из контейнера	Эксплойт ядра/рантайма → доступ к хосту	gVisor (перехват сисколлов) / Firecracker (микро-VM)

10.2. Слои изоляции: чем сильнее, тем лучше

Термины: gVisor, Firecracker, эфемерность, blast radius

gVisor — «прослойка» между контейнером и ядром хоста: перехватывает системные вызовы в пользовательском пространстве, резко сужая поверхность атаки на ядро. **Firecracker** — минималистичный гипервизор «микро-VM» с быстрым стартом (десятки мс), дающий *аппаратную* изоляцию почти по цене контейнера. **Эфемерность** — среда живёт одну попытку и уничтожается. **Blast radius** («радиус поражения») — насколько далеко может распространиться ущерб от одной скомпрометированной песочницы; цель — свести к одной попытке.

Рекомендация под ваш масштаб: **эфемерная микро-VM (Firecracker или gVisor-контейнер) на каждую попытку**, с уникальным сетевым namespace, allowlist-ом egress строго на хост задачи, cgroups-лимитами и «убийцей» по таймауту. Никаких API-ключей фермы, учёток облака или доступа к доске *внутри* песочницы — агент общается с фермой только через узкий, проверяемый канал (сдаёт находки, получает задачу), а не имеет прямого доступа к её потрохам.

10.3. Prompt injection — специфическая для агентов угроза

Содержимое задачи (README, строки в бинарнике, ответ сервиса) — это **недоверенные данные**, а агент читает их как часть наблюдений. Злонамеренный (или просто «хитрый») текст может пытаться перенаправить агента. Практика защиты: (1) чётко **разделять «инструкции системы» и «данные наблюдений»** на уровне промпта; (2) не давать агенту действий с необратимыми внешними последствиями без проверки; (3) для фермы это менее критично, чем в клиентских ассистентах (нет доступа к почте/деньгам пользователя), но **egress-контроль и отсутствие секретов** нейтрализуют большинство последствий.

Красная линия

Никогда не запускайте недоверенные CTF-артефакты в основной среде фермы, в общей сети или на хосте с доступом к вашим ключам/данным. Одна «RE»-задача, оказавшаяся настоящим малварём, при слабой изоляции может скомпрометировать всю ферму. Изоляция — это то, на чём *нельзя* экономить ради скорости старта.

ЗАМКНУТЬ КОНТУР

11 Наблюдаемость, оценка и самоулучшение

Ферма без измерений — это чёрный ящик, который тратит деньги. А главное — «золотой сигнал» (флаг) позволяет не просто наблюдать, но и **обучать** ферму на её собственном опыте. Это тот контур, который со временем отрывает вас от конкурентов.

11.1. Трейсинг каждого шага агента

Каждая траектория логируется как **трейс**: последовательность мысль→действие→наблюдение, с токенами, стоимостью, задержками и результатом. Инструменты класса Langfuse (или собственный слой поверх OpenTelemetry) дают: реплей траектории (что именно делал агент), разбор провалов, сравнение версий промптов/инструментов. Без этого вы не сможете отлаживать рой — поведение сотен агентов иначе непостижимо.

11.2. Метрики, которые действительно важны

Метрика	Что показывает	Как используется
Cost-per-flag	Ресурсы на один взятый флаг	Главный KPI эффективности; цель — снижать
Time-to-flag	Скорость конвертации задачи в очки	Критично в live против динам. скоринга
Solve-rate по категориям	Где рой силён/слаб	Куда вкладывать инструменты/демонстрации
Escalation-rate	Доля эскалаций в Pro	Калибровка порога каскада (3.2)
Tool success-rate	Насколько надёжны инструменты	Где чинить ACI (2.3, 4.2)
Stall/cancel-rate	Сколько траекторий «утонуло»	Настройка watchdog и бюджетов

11.3. Банк решений и RAG-контур

Каждый успешный (и показательно неуспешный) трейс — актив. Из них собирается **банк решений**: райтапы, шаблоны атак, «как выглядела похожая задача и что сработало». Агенты обращаются к нему через RAG на этапе планирования. Это дёшево умножает силу Flash-агентов: модель среднего уровня + хорошая память известных техник часто бьёт более сильную модель без памяти — особенно на «шаблонных» категориях (2.2).

11.4. Самоулучшение: от логов к дообучению

Термины: RFT и DPO

RFT (Reinforcement Fine-Tuning) — дообучение модели с подкреплением по сигналу награды; в CTF награда «бесплатна» и чиста — это флаг. **DPO (Direct Preference Optimization)** — более простой способ подстроить модель под «удачные vs неудачные» траектории без полноценного RL-контура. Оба возможны *только* при доступе к весам — то есть при self-host открытой V4.

Ступени зрелости контура улучшения: **(1)** калибровка гиперпараметров (порог эскалации, бюджеты) по метрикам — дёшево и сразу; **(2)** майнинг лучших траекторий в демонстрации (few-shot) — почти бесплатно, ощутимый эффект; **(3)** офлайн-дообучение Flash (DPO/RFT) на своём «золотом» датасете решённых задач — дороже, но именно здесь окупается *открытость весов DeepSeek-V4* и self-host. Со временем ваш дообученный Flash становится дешевле *и* лучше стокового на вашем распределении задач — это и есть устойчивое преимущество фермы.

Мнение: где «ров» вашей фермы

Скопировать архитектуру может каждый. Что скопировать нельзя — это ваш **накопленный банк решений и дообученные под ваши задачи веса**. Поэтому с первого дня *логируйте всё* и складывайте траектории: даже если дообучение вы включите через полгода, данные для него нужно копить с самого начала. Обесценивание CTF под натиском ИИ (раздел 2.2) означает, что выиграет не тот, у кого «модель умнее», а тот, у кого лучше *контур накопления опыта*.

КАК СТРОИТЬ

12 Дорожная карта внедрения

Не стройте всё сразу. Каждая фаза даёт работающую систему и снимает конкретный риск. Порядок выбран так, чтобы **раньше проверить самое неопределённое** (умеет ли агент вообще решать) и позже вкладываться в дорогое (self-host, дообучение).

Фаза	Что делаем	Состав	Снимаемый риск / результат
0 MVP	Один агент, одна категория, всё через API	ReAct + базовый тулкит + песочница + проверка флага	**«Агент вообще решает?» → цифры бенчмарка**
1 Рой	Много агентов на API, доска, триаж, все категории	Blackboard + очередь + специализация + каскад Flash→Pro (API)	«Координация даёт выигрыш?» → рой > одиночки
2 Инфра	Гибридный инференс и устойчивость	Self-host Flash (SGLang) + модельный шлюз + Temporal/Restate	Снижение cost-per-flag; устойчивость к сбоям
3 Live	Планировщик реального времени	Скорборд-коннектор + приоритеты/knapsack + бюджеты + «дожатие»	**Готовность к «боям»: очки к дедлайну**
4 Ров	Контур самоулучшения	Трейсинг + банк решений (RAG) + калибровка + DPO/RFT Flash	Устойчивое преимущество, снижение затрат

Правило перехода между фазами

Переходите на следующую фазу **по метрике, а не по календарю**. Нет смысла в дорогом self-host (фаза 2), пока рой на API (фаза 1) не показал, что вы упираетесь в стоимость или rate limits. Преждевременная оптимизация инфраструктуры — самая частая и дорогая ошибка таких проектов.

12.1. Что можно собрать за первые выходные

Фаза 0 полностью реализуема на API + `docker compose`: один ReAct-агент (любой каркас: SWE-agent-подобный, собственный цикл), песочница-контейнер с тулкитом под одну категорию (начните с *web* или *forensics* — там выше отдача), проверка флага и трейсинг. Это даст вам честные цифры и «пощупать» реальные узкие места до того, как вы вложитесь в GPU.

СБОРКА ВОЕДИНО

13 Рекомендация под ваш кейс и риски

Соберём всё сказанное под ваши ответы: **живые соревнования**, **малый-средний масштаб**, **все сценарии хостинга рассмотрены**, **комплексная глубина**. Ниже — итоговая референсная архитектура и честный список рисков.

13.1. Референсный стек (то, что я бы строил на вашем месте)

Слой	Выбор	Почему
Модель-ядро	DeepSeek-V4-Flash (воркеры) + V4-Pro (эскалация)	Цена/качество + каскад (разделы 3, 7)
Хостинг	Гибрид: Flash self-host + Pro API (сценарий C)	Низкая предельная цена + контроль + гибкость пика
Движок инференса	SGLang (осн.) + vLLM (батч/резерв)	RadixAttention под общие префиксы роя (9.3)
Модельный шлюз	Прокси-слой (LiteLLM-подобный) или свой	Роутинг Flash/Pro, фолбэк, учёт стоимости (6.4)
Оркестрация	Temporal (или Restate)	Durable-workflow для долгих агентов (6.3)
Очередь	Redis Streams → NATS при росте	Минимум частей на старте (6.2)
Координация роя	Blackboard (Postgres+Redis) + менеджер-воркеры + work-stealing	Переиспользование находок, балансировка (раздел 5)
Песочница	Эфемерные Firecracker/gVisor на попытку	Недоверенный код — несущая безопасность (раздел 10)
Данные	Postgres + Redis + MinIO + pgvector	Один compose закрывает все роли (6.5)
Наблюдаемость	Langfuse-класс + метрики cost/time-per-flag	Отладка роя и топливо для улучшения (11)
Улучшение	Банк решений (RAG) → калибровка → DPO/RFT позже	«Ров» фермы на открытых весах (11.4)

13.2. Три вещи, на которых нельзя экономить

① Инструменты и ACI

Прирост от интерактивных инструментов и хорошего интерфейса больше, чем от апгрейда модели (2.3).

③ Трейсинг с первого дня

Данные для будущего дообучения нужно копить *сейчас*, даже если обучать будете позже (11.4).

② Изоляция

Одна недоверенная задача при слабой песочнице компрометирует всю ферму (раздел 10).

И одно, на чём экономить нужно

Не разворачивайте V4-Pro self-host и не строите GPU-автоскейл под редкие «бои» — это деньги на ветер на вашем масштабе.

13.3. Риски и открытые вопросы

Риск	Суть	Что делать
Правила соревнований	Многие CTF ограничивают/запрещают полную автономию или заводят отдельные AI-дивизионы	Читать ToS каждого события; играть в разрешённом дивизионе
Цена/доступность V4-Pro	Тарифы Pro в источниках расходятся и могут меняться	Абстракция провайдера (6.4); сверять цену перед «боем»
«Хвост» сложности	Hard-pwn и продвинутая крипта всё ещё низко решаются (2.1)	Не обещать себе 100%; держать альт-эскалацию, спец-тулинг
Runaway-затраты	Зацикленные агенты жгут токены/GPU	Жёсткие бюджеты, watchdog, admission control (раздел 9)
Операционная нагрузка self-host	MoE-инференс, отказоустойчивость, обновления	Начать с API (фаза 1), self-host — по метрике
Vendor lock-in весов	Жёсткая привязка к одному семейству моделей	Модельный шлюз + возможность подмешать другие модели

Итоговое мнение в одном абзаце

Ваша базовая интуиция верна: DeepSeek-V4 — рациональное, экономичное ядро для роя. Но выигрыш придёт **не от количества агентов и не от «модели помощнее», а от трёх вещей**: (1) *каскада* дешёвого Flash с редкой Pro-эскалацией, (2) *качества инструментов, песочницы и координации* (куда идёт большая часть инженерии), и (3) *контура накопления опыта* (банк решений + дообучение на открытых весах). Стройте гибрид, начинайте с MVP на API, измеряйте cost-per-flag и time-to-flag с первого дня — и наращивайте инфраструктуру строго по метрикам, а не заранее.

Ответственность

Автономные наступательные агенты — инструмент двойного назначения. Применяйте ферму только на легитимных площадках, с явного разрешения и в рамках правил. Ответственность за использование лежит на операторе.

ПРИЛОЖЕНИЕ А

A Глоссарий «трудных» терминов

Все специальные термины, встречавшиеся в тексте, в одном месте. Сгруппированы по темам.

CTF и соревнования

CTF Capture the Flag
Соревнование по практической ИБ: найти секретную строку-«флаг» в задании.

Jeopardy / task-based
Формат с доской независимых задач по категориям и стоимостью в очках.

Attack-defense
Альтернативный формат: команды атакуют сервисы друг друга в реальном времени.

Флаг flag
Строка-доказательство решения, обычно `flag{...}`. Даёт чистый сигнал успеха.

Динамический скоринг
Стоимость задачи падает по мере роста числа решивших команд.

Ground-truth reward
«Золотой» однозначный сигнал награды (взят флаг или нет).

Embarrassingly parallel
Задачи, которые тривиально распараллеливаются без взаимной координации.

Категории задач

Pwn binary exploitation
Эксплуатация ошибок работы с памятью (переполнения, ROP, heap).

Reverse / RE
Обратная разработка бинарников для понимания их логики.

Crypto
Атаки на слабые/неправильно применённые криптосхемы.

Forensics
Анализ дампов памяти, трафика, файлов; стеганография.

Модель и инференс

MoE Mixture-of-Experts
Много «экспертов», но на токен активна лишь часть (роутер выбирает top-k).

Активные vs всего параметров
Активные → скорость/цена токена; всего → сколько VRAM под веса.

KV-кэш
Память под уже обработанный контекст; для 1M-контекста огромен.

PagedAttention
vLLM: KV-кэш страницами — больше параллельных запросов в VRAM.

RadixAttention
SGLang: KV-кэш в префиксном дереве — переиспользование общих префиксов.

Prefix-кэш
Переиспользование общего начала промпта между запросами — экономия.

TTFT time-to-first-token
Задержка до первого токена ответа; определяет «отзывчивость».

Throughput
Пропускная способность инференса (токенов/сек).

Continuous batching
Динамическое доукомплектование батча на GPU новыми запросами.

TP / EP
Тензорный / экспертный параллелизм — способы «размазать» модель по GPU (EP — для MoE).

Квантизация (FP4)
Снижение точности весов ради памяти/скорости; V4 использует FP4-QAT.

Speculative decoding

Агент и рой

ReAct Reason+Act
Цикл «мысль → действие инструментом → наблюдение → …».

ACI Agent-Computer Interface
«UX для модели»: как представлены инструменты и наблюдения.

IAT Interactive Agent Tools
Интерактивные инструменты (отладчик, сетевое соединение) — дают скачок в pwn/RE.

Blackboard
Общая «доска» находок, через которую координируется рой.

Manager–workers
Лёгкий распределитель кладёт задачи в очередь, воркеры их разбирают.

Work-stealing
Простаивающий воркер «крадёт» задачи у перегруженного — балансировка.

Contract-net
«Тендер» на задачу: агенты делают ставки, задача уходит лучшему.

Model cascade
Каскад: дешёвая модель решает сама, дорогая подключается по эскалации.

Ускорение декодирования за счёт «черновой» модели-предсказателя.

Инфраструктура и надёжность

Durable execution
Движок пишет каждый шаг, чтобы продолжить после сбоя с места падения.

Idempotency
Свойство «повтор не создаёт дубля побочного эффекта».

Heartbeat
Периодический «пульс» GPU-задачи; пропал — задачу переназначают.

Backpressure
Обратное давление: при перегрузке система тормозит приём, а не падает.

Admission control
Контроль допуска: при дефиците отклоняются низкоприоритетные задачи.

Model gateway
Единый шлюз к моделям: роутинг, фолбэк, учёт стоимости.

Безопасность и обучение

gVisor
Перехват сисколлов в user-space — сужает поверхность атаки на ядро.

Firecracker
Минималистичный гипервизор «микро-VM» с быстрым стартом.

Эфемерность
Среда живёт одну попытку и уничтожается.

Blast radius
«Радиус поражения» от одной скомпрометированной песочницы.

Egress-контроль
Ограничение исходящего трафика песочницы (allowlist на цель).

Prompt injection
Вредоносные инструкции в недоверенных данных, читаемых агентом.

RAG

Генерация с опорой на поиск по базе знаний (банк решений).

RFT / DPO

Дообучение с подкреплением / прямая оптимизация предпочтений (нужны веса).

Cost/Time-per-flag

Главные KPI: ресурсы и время на один взятый флаг.

ПРИЛОЖЕНИЕ В

В Источники

Материалы, использованные при подготовке отчёта (проверено в сентябре 2026). Технические цифры по моделям и бенчмаркам следует перепроверять на момент запуска — рынок и тарифы быстро меняются.

DeepSeek-V4: спецификации и цены

- DeepSeek V4 — V4-Pro (1.6T) и V4-Flash (284B): https://deepseek.ai/deepseek-v4
- DeepSeek V4 — specs, pricing, how to run: https://www.morphllm.com/deepseek-v4
- DeepSeek V4 Flash — 284B MoE, 1M context: https://www.morphllm.com/deepseek-v4-flash
- DeepSeek V4 release (1.6T, 1M context, open weights): https://datanorth.ai/news/deepseek-releases-v4-open-source-1m-context
- DeepSeek V4 (2026) benchmarks: https://www.nxcode.io/resources/news/deepseek-v4-release-specs-benchmarks-2026

Автономные CTF-агенты: бенчмарки и результаты

- NYU CTF Bench (NeurIPS'24 D&B): https://nyu-llm-ctf.github.io/
- EnIGMA: Interactive Tools Substantially Assist LM Agents (arXiv 2409.16165): https://arxiv.org/html/2409.16165.pdf
- InterCode-CTF (обзор): https://www.emergentmind.com/topics/intercode-ctf
- CTFusion: CTF-based benchmark for LLM agents via MCP (arXiv 2605.11504): https://arxiv.org/html/2605.11504v1
- «The Death of the CTF: How Agentic AI Is Reshaping Competitive Hacking» (Suzu Labs): https://suzulabs.com/suzu-labs-blog/the-death-of-the-ctf-how-agentic-ai-is-reshaping-competitive-hacking

Инференс и оркестрация

- vLLM vs SGLang 2026 (RadixAttention vs PagedAttention, MoE, H100): https://www.spheron.network/blog/vllm-vs-sglang-2026/
- AI Agent Workflow Orchestration (Temporal / Inngest / Restate) on GPU cloud: https://www.spheron.network/blog/ai-agent-workflow-orchestration-temporal-inngest-restate-gpu-cloud/

Подготовлено для проекта **slopi-farm**. Документ носит инженерно-аналитический характер и не является юридической консультацией; проверяйте правила конкретных соревнований и применимое законодательство до запуска фермы.